

ROOFLANG: ENABLING AI-DRIVEN ARCHITECTING OF LLM INFERENCE SYSTEMS

Ziyue Yang^{1*} Yuting Jiang¹ Lei Qu¹ Peng Cheng²

¹Shanghai Xingyunzhi Artificial Intelligence Institute, Shanghai, China

²Microsoft Research, Redmond, WA, USA

ABSTRACT

AI is beginning to make substantive contributions to LLM inference optimization. Existing AI optimizations are predominantly profiling-based. Profiling-bound feedback confines the search to the capabilities and performance of an existing software stack, preventing a fundamentally better architecture of LLM inference systems from being identified. To enable the AI-driven LLM inference system architecting loop, we argue that a general workload representation, a verifiable mutation space, and an implementation-independent evaluator are required. We present the ROOFLANG domain-specific language (DSL) that provides these features. In our evaluation, ROOFLANG reveals that DeepSeek V4-series models could achieve $3.5\text{--}39.5\times$ higher peak decode throughput than other representative models. This gap is disproportionate to their total parameter counts and arises largely from compact KV-cache designs that support larger batches and reduce memory traffic. A persistent optimizer agent further discovered several new architectures that improved both throughput and interactivity of DeepSeek V4 Pro on NVIDIA B300 by 6.23–50.1%.



<https://github.com/yzygitzh/rooflang>



<https://yzygitzh.github.io/rooflang>

1 INTRODUCTION

AI systems are beginning to take on substantive parts of large language model (LLM) inference optimization. Recent agents and automated workflows optimize kernels, compiler-generated code, framework configurations, and serving paths, turning implementation-level performance engineering into a machine-actionable task (Ouyang et al., 2025; Xing et al., 2026; Andrews & Witteveen, 2025; Gai et al., 2026; MIT HAN Lab, 2026; Yeon et al., 2026). This emerging capability is an important component of recursive self-improvement (RSI) of LLMs (Schmidhuber, 2007; Zhang et al., 2026).

Most existing approaches organize this work around profiling. Profiling identifies hotspots in a current implementation and provides a clear verifier for whether a local change helps (Andrews & Witteveen, 2025; Gai et al., 2026; MIT HAN Lab, 2026; Unger et al., 2022). This feedback loop is productive, but its potential is constrained by the software stack being profiled. A fundamentally different architecture may better satisfy the serving objective yet remain unrealizable with the current stack because required features are absent or their implementations are insufficiently optimized.

It would be desirable for AI to architect LLM inference systems from scratch and continuously optimize them. Closing such an autonomous loop requires a general representation of LLM workloads, a verifiable action space for architecting, and a mechanism to evaluate resulting architectures and provide feedback. These actions can organize computation and data through graph transformations, sharding, placement, communication, and memory choices for a target workload and hardware platform, potentially yielding more fundamental gains than tuning a fixed implementation surface.

To realize this vision, we present the ROOFLANG domain-specific language (DSL), which provides these features. RoofLang represents workloads and hardware as graphs, provides semantics-

*Correspondence to: Ziyue Yang <yzylivezh@hotmail.com>.

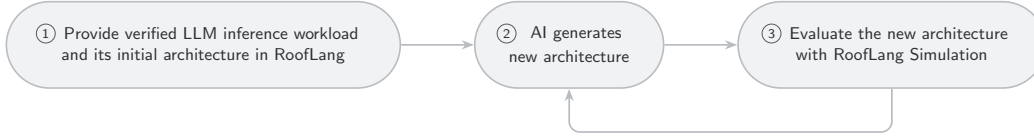


Figure 1: AI-driven autonomous architecting loop enabled by RoofLang.

preserving graph transformations and placement primitives as architecting actions, and evaluates the resulting organizations with a roofline-based discrete-event simulator. The evaluator exposes compute, memory, network, dependency, contention, lifetime, and capacity costs while reducing interference from software-stack implementation details. Given a verified initial workload and architecture, an AI agent can propose new valid architecting actions and iteratively improve them using simulation feedback, closing the autonomous loop to explore meaningful architecting choices, as demonstrated in Figure 1.

Our evaluation reveals two salient findings. First, DeepSeek V4-series models could achieve 3.5–39.5× higher peak decode throughput than other representative models. Total parameter counts alone cannot explain this gap. Instead, the advantage arises largely from compact KV-cache designs that support larger batches and reduce memory traffic. Second, a persistent optimizer agent discovered several new architectures that improved both throughput and interactivity of DeepSeek V4 Pro on NVIDIA B300 by 6.23–50.1%. Together, these results expose model-level design insights and show that RoofLang can guide AI toward useful system architectures.

Our main contributions are as follows:

- **A practical DSL for autonomous LLM inference system architecting.** RoofLang combines general graph representations, a semantics-preserving action space, and a roofline-based evaluator in an AI-accessible loop.
- **An analysis of state-of-the-art models and architecting choices.** We study how model, workload, and hardware characteristics affect graph organization, placement, sharding, memory, communication, and their trade-offs.
- **An evaluation of autonomous architecting by AI.** We compare AI-driven exploration with heuristic grid search and examine how an agent uses simulation feedback to generate meaningful architecting choices.

2 ROOFLANG DESIGN

2.1 OVERVIEW

Architecting an LLM inference system can be viewed as organizing computation and data to optimize a target objective. Autonomous architecting can be realized with the following three elements:

- **A general workload representation.** A format sufficiently general to describe LLM inference workloads without committing to a specific software stack.
- **Verifiable architecting actions.** This capability is necessary to ensure that architecting actions made by autonomous processes are valid.
- **An architecture evaluator.** A mechanism for evaluating the quality of an architecting action while minimizing interference from factors unrelated to architecting, such as software-stack implementation details.

Given these elements and a verified input workload, an AI agent can generate valid architecting actions, whose evaluation result then provides feedback for revising those actions, enabling an autonomous loop for AI-driven architecting.

Compute graphs provide a general workload representation, as widely used in modern AI frameworks (Abadi et al., 2016). Verified graph optimizers such as Unity demonstrate how graph substitutions can preserve semantics while changing computation and parallelization (Unger et al., 2022).

Our approach combines graph-based workload and hardware representations, constrained transformation and placement primitives, and roofline-based simulation (Williams et al., 2009). This combination evaluates architecting choices from analytical workload attributes and hardware specifications without requiring profiled operator implementations.

We therefore present the RoofLang DSL, which provides these features for AI-driven architecting of LLM inference systems. Figure 2 gives an overview. A correct and externally supplied compute graph is first rewritten through semantics-preserving transformations. Placement combines the transformed compute graph with a hardware graph by assigning operators and tensors to hardware resources. Further semantics-preserving transformations can refine the transformed and tagged compute graph, including its communication realization. Finally, the transformed and tagged compute graph and the hardware graph are evaluated by roofline-based simulation, which emits a trace as feedback for subsequent architecting actions.

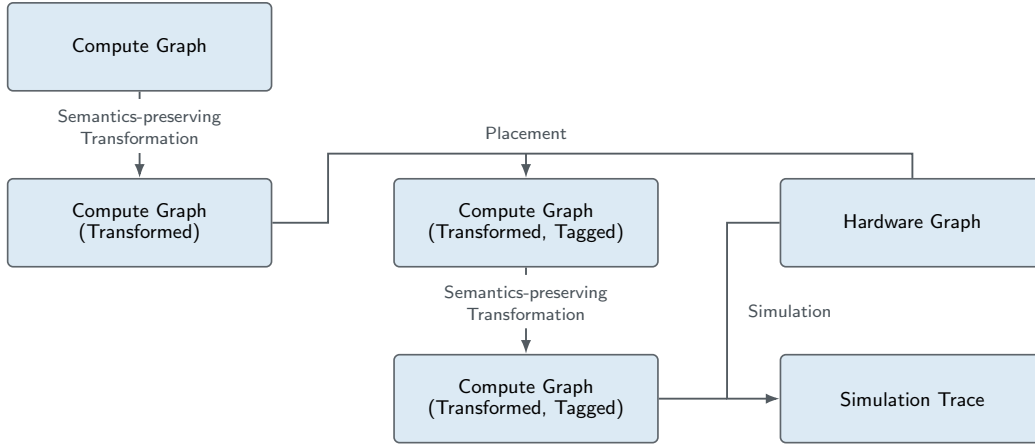


Figure 2: RoofLang DSL overview.

The following subsections introduce the different components of the RoofLang DSL.

2.2 COMPUTE GRAPH

The compute graph is an input to RoofLang and provides a way to generally describe an LLM inference workload. It must be supplied externally, and its provider is responsible for guaranteeing that it faithfully and correctly represents the workload.

Each node in the compute graph is a compute node that represents an operation, e.g., attention, GEMM, etc. A node specifies its input and output tensors and has analytical workload attributes such as tensor shapes and dtypes, FLOPs, and data movement. Compute nodes are classified as side-effect-free or side-effecting: a side-effect-free node is observable only through its output tensors, whereas a side-effecting node may also modify externally observable state. Each edge in the compute graph represents a data dependency from an output tensor of one operation to an input tensor of another.

2.3 HARDWARE GRAPH

Another input to RoofLang is the hardware graph, which provides a way to generally describe the target hardware platform for LLM inference. It specifies the available compute and memory components and the fabrics that connect them. The graph is supplied externally, and its provider is responsible for guaranteeing that it faithfully and correctly represents the target hardware.

Each node in the hardware graph is either a compute device (e.g., CPUs, accelerators, NICs, switches, etc.) that has peak FLOPs for each supported precision or a memory device (e.g., DRAM, HBM, SSDs, etc.) that has a specified capacity. Each edge in the hardware graph represents a fabric

Table 1: Placement primitives.

Primitive	Function
<code>operator.set_compute</code>	Assigns an operator to the compute device and, optionally, the underlying stream/queue on which it executes.
<code>tensor.set_memory</code>	Assigns a tensor to the memory device in which it is stored.

Table 2: Semantics-preserving transformation primitives.

Primitive	Function
<code>graph.add_control_edge</code>	Adds a happens-before dependency between two kernels.
<code>graph.remove_control_edge</code>	Removes a control edge between two kernels.
<code>graph.insert_identity</code>	Insert an identity operator between two kernels.
<code>graph.remove_identity</code>	Remove an identity operator between two kernels.
<code>graph.fuse_kernels</code>	Replaces a subgraph with an operator denoting the same composition at the external boundary, using pre-defined rules, e.g., qk-norm-rope fusion.
<code>graph.split_kernel</code>	Replaces an operator with an equivalent partitioned or replicated subgraph and proper communication operators, using pre-defined rules. Examples for splitting a $(m, k) \times (k, n)$ GEMM are: • split-by-m: scatter–GEMMs–gather • split-by-k: scatter–GEMMs–reduce • split-by-n: broadcast–GEMMs–gather
<code>graph.dup_kernel</code>	Replicates a side-effect-free operator whose outputs are broadcasted, i.e., operator-broadcast $\Rightarrow$ broadcast-operators.
<code>graph.dedup_kernel</code>	Merge multiple same side-effect-free operators which accept broadcasted inputs, i.e., broadcast-operators $\Rightarrow$ operator-broadcast.
<code>graph.merge_gather_scatter</code>	Replaces a gather-scatter subgraph with per-operator-pair direct data edges if splitting dimensions of both operators are the same, else with an all-to-all operator.
<code>graph.merge_gather_broadcast</code>	Replaces a gather-broadcast subgraph with an all-gather operator.
<code>graph.merge_reduce_scatter</code>	Replaces a reduce-scatter subgraph with a reduce-scatter operator.
<code>graph.merge_reduce_broadcast</code>	Replaces a reduce-broadcast subgraph with an all-reduce operator.

connection (e.g., PCIe, Ethernet, etc.) and has communication properties, including bandwidth and latency.

2.4 PLACEMENT

Placement lowers a compute graph onto a hardware graph, tags operators with compute devices and tensors with memory devices, so that it can be evaluated by the simulator. This is realized by two placement primitives summarized in Table 1. Note that placement is also semantics-preserving.

2.5 SEMANTICS-PRESERVING TRANSFORMATION

RoofLang implements semantics-preserving transformations through a set of primitives over the compute graph, summarized in Table 2. Each primitive applies a pre-defined, structurally constrained rewrite whose stated preconditions preserve the modeled observations.

Executing a transformation updates the analytical attributes of the affected operators and tensors. For example, operator FLOPs and bytes transferred, as well as tensor shapes and communication volumes, are recomputed for the transformed subgraph. Placement and semantics-preserving transformation can be composed to express common LLM inference architecting actions. For example, RoofLang can encode the following actions, with the relevant primitives shown explicitly:

- **Prefill–decode (PD) disaggregation:** the primitives `operator.set_compute` and `tensor.set_memory` assign prefill and decode computation, together with exchanged state, to distinct compute and memory devices (Zhong et al., 2024).
- **KV-cache lifecycle:** `graph.dup_kernel` and `graph.dedup_kernel` can be used to control the granularity of KV recomputation and caching, while identity operators such as `ToNvme`, `ToDram`, and `ToHbm` can be inserted or removed with `graph.insert_identity` and `graph.remove_identity` to describe different KV-cache memory-tier policies (Sheng et al., 2023; Kwon et al., 2023).
- **Sharding:** `graph.split_kernel`, `operator.set_compute`, `tensor.set_memory`, and communication merging primitives can be used to realize typical model sharding mechanisms, including tensor parallelism (TP), context parallelism (CP), data parallelism (DP), expert parallelism (EP), and pipeline parallelism (PP) (Narayanan et al., 2021; Jacobs et al., 2023; Lepikhin et al., 2020).

2.6 SIMULATION

The simulator executes the transformed and tagged compute graph as a topology-aware discrete-event simulation (DES). It obtains a topological order, releases a kernel when all data and control predecessors have completed and its assigned stream is available, and schedules start and completion events on a priority queue. Each completion updates successor readiness, tensor liveness, and resource occupancy, so the trace represents dependency- constrained execution rather than a static sum of operator costs.

Memory footprints of inputs, outputs, and weights are tracked with operator entry and exit events. The simulator reports the peak live bytes for each memory device and raises an out-of-memory diagnostic when a capacity is exceeded.

For an operator K placed on compute device d , RoofLang models compute, local-memory, and networking costs, respectively.

For compute cost, with q indexing dtypes, $F_{K,q}$ denoting FLOPs, and $P_{d,q}$ denoting the device peak FLOP/s at dtype q ,

$$T_{\text{compute}}(K, d) = \sum_q \frac{F_{K,q}}{P_{d,q}}. \quad (1)$$

For local-memory cost, including tensor reads from and writes to local memory,

$$T_{\text{memory}}(K, d) = \sum_{t \in \mathcal{R}_K} \frac{B_{K,t}^{\text{read}}}{\beta_{d,t}^{\text{read}}} + \sum_{t \in \mathcal{W}_K} \frac{B_{K,t}^{\text{write}}}{\beta_{d,t}^{\text{write}}}, \quad (2)$$

where the byte terms include the modeled input, weight, and output read/write fractions.

For networking cost, including remote accesses and explicit communication, data are routed through the directed hardware fabrics. The corresponding network cost is

$$T_{\text{network}}(K) = \alpha_K + \begin{cases} \max_{\ell \in \mathcal{L}_K} \frac{B_{K,\ell}}{\beta_\ell}, & \text{remote tensor access,} \\ \frac{V_K}{B_{\text{eff},K}}, & \text{collective communication,} \end{cases} \quad (3)$$

with latency α_K , link bytes $B_{K,\ell}$, link bandwidth β_ℓ , and collective volume V_K and effective participant bandwidth $B_{\text{eff},K}$. In the no-contention baseline, the operator duration is the bottleneck component, assuming ideal full overlap among compute, local-memory, and networking costs (with byte and rate units chosen consistently),

$$T_K = \max \{T_{\text{compute}}, T_{\text{memory}}, T_{\text{network}}\}. \quad (4)$$

Table 3: Models used in RoofLang evaluation.

Model	Parameters	Layers	Attention	FFN
DeepSeek V4 Flash	284BA13B	43	CSA + HCA	1 shared and top-6 of 256 routed experts
GLM-5.3	744BA40B	78	DSA + MLA	3 dense layers, 1 shared and top-8 of 256 routed experts
DeepSeek V4 Pro	1.6TA49B	61	CSA + HCA	1 shared and top-6 of 384 routed experts
Kimi K3	2.8TA104B	93	KDA + MLA	1 dense layer, 2 shared and top-16 of 896 routed experts

Table 4: Model-specific numerical precision configuration in RoofLang evaluation.

Model	KV	Attention	FFN
DeepSeek V4 Flash	FP8 main cache and FP4 index cache	FP8	FP4
GLM-5.3	FP8 main cache and FP8 index cache	BF16	FP8
DeepSeek V4 Pro	FP8 main cache and FP4 index cache	FP8	FP4
Kimi K3	FP8 MLA cache and BF16 KDA states	FP8 MLA and BF16 KDA	FP4

Collective communication uses a theoretical $\alpha + \beta$ model rather than a packet-level schedule (Hockney, 1994). The α term is the maximum path latency among the participating ranks, i.e., the weighted diameter of their data propagation graph. The β term is an aggregate bandwidth estimated heuristically from the hardware topology. Its effective value is further controlled by the fabric resource-sharing mechanism described below, so concurrent transfers reduce the available bandwidth according to contention.

The DES models resource sharing while events are active. Operators consuming a compute device or a fabric receive normalized, capacity-weighted shares: if c_i is an operator’s resource cap and A_r is the active set consuming resource r , its share is $c_i / \max(1, \sum_{j \in A_r} c_j)$. When sharing occurs among multiple operators, the simulated cost during the sharing interval extends proportionately until the set of sharing operators changes. This gives a reasonable estimation of scenarios where multiple operators share a single resource.

3 EVALUATION

3.1 ROOFLANG EFFICACY ON REPRESENTATIVE LLMs AND HARDWARE

3.1.1 SETUP

Models. We evaluate four representative models: DeepSeek V4 Flash, GLM-5.3, DeepSeek V4 Pro, and Kimi K3 (DeepSeek-AI et al., 2026; GLM-5-Team et al., 2026; Z.ai, 2026; Kimi Team et al., 2026). Tables 3 and 4 summarize their model structures and their numerical precision configurations, respectively. For simplicity, we omit KV-cache appending and speculative-decoding structures from all four models, as well as the multimodal components of Kimi K3.

Hardware. Table 5 summarizes four representative hardware platforms: NVIDIA H200, NVIDIA GH200, NVIDIA B300, and NVIDIA GB300. We assume NVLink for scale-up networking and InfiniBand for scale-out networking. For NVIDIA H200 and NVIDIA GH200, they cannot do native FP4, and we assume they can emulate FP4 as fast as FP8. The modeled peak rates, memory capacities, and interconnect parameters are derived from NVIDIA specifications (NVIDIA, n.d.; 2026a;b; 2025).

Workload. Table 6 summarizes the workload space evaluated, including stages, contexts, GPU counts, parallelism dimensions, and batch sizes. For GPU counts, we select 8 and 64 to probe the efficacy of larger scale-up networking scope. For context length, we select 64K as it is the dominant long-context regime according to a recent study Lee et al. (2026), and we select 1M as it is the maximum supported by the evaluated models (DeepSeek-AI et al., 2026; Z.ai, 2026; Kimi Team et al., 2026). We adopt context parallelism (CP) instead of tensor parallelism (TP) because sequence-wise sharding is better suited to long contexts and avoids the KV-cache replication that TP can induce Lee et al. (2026). We further heuristically let $DP \times CP = EP$, referencing the large-scale inference configuration in the DeepSeek-V3 report DeepSeek-AI et al. (2024). For batch size, we sweep 2^n from $n = 0$ and stop at the first OOM point.

Table 5: Hardware platforms used in RoofLang evaluation.

Platform	FP4 TFLOP/s	FP8 TFLOP/s	BF16 TFLOP/s	TF32 TFLOP/s	HBM GB	HBM BW GB/s	NVLink BW GB/s	NVLink max scope	InfiniBand BW GB/s
NVIDIA H200	1,979	1,979	989	495	144	4,800	450	8	50
NVIDIA GH200	1,979	1,979	989	495	144	4,900	450	256	50
NVIDIA B300	13,500	4,500	2,250	1,125	288	7,750	900	8	100
NVIDIA GB300	15,000	5,000	2,500	1,250	288	8,000	900	72	100

Table 6: Workload configuration search space in RoofLang evaluation.

Configuration	Values
Stage	prefill, decode (1-step)
Context	64K, 1M
GPU counts	8, 64
Parallelism	DP, CP, PP, EP, where $EP \times PP = DP \times CP \times PP = n_{\text{GPUs}}$
Batch size	2^n until OOM, $n \geq 0$

Simulation. We run the discrete-event simulation (DES) as described in Section 2.6. Across the overall execution timeline, we assume that computation and communication overlap perfectly through fine-grained overlapping (DeepSeek-AI et al., 2024), and derive the final throughput and interactivity from the overlapped timeline. We further assume that routed tokens are ideally balanced across MoE experts.

3.1.2 RESULTS

Figures 3 and 4 plot the resulting throughput–interactivity Pareto frontiers for prefill and decode, respectively.

Frontier shape. Across models, contexts, hardware platforms, and GPU counts, the curves exhibit the expected throughput–interactivity shape: higher throughput coincides with lower interactivity, and vice versa.

Frontier endpoints. The left endpoints for prefill correspond to large-batch, high-arithmetic-intensity configurations, and their throughput values are primarily bounded by model compute FLOPs divided by GPU FLOP rates. The right endpoints of decode correspond to small-batch, interactivity-oriented configurations, and their interactivity values are primarily bounded by the activated-parameter bytes per step divided by HBM bandwidth.

Decode throughput. Among the plotted curves for 64 NVIDIA GB300 GPUs, at 64K-context and 1M-context, respectively, DeepSeek V4 Flash reaches $9.6\times$ and $16.7\times$ the peak decode throughput of GLM-5.3. The corresponding ratios over Kimi K3 are $25.2\times$ and $39.5\times$. For DeepSeek V4 Pro, the 64K and 1M ratios are $3.5\times$ and $5.3\times$ over GLM-5.3, and $9.3\times$ and $12.4\times$ over Kimi K3. These gaps arise in large part from per-decode-token memory overhead. For batch size B , we calculate this overhead as

$$M_{\text{decode}} = \frac{P + BT_{\text{KV}}}{B}, \quad (5)$$

where P is the encoded model-weight size and T_{KV} is the per-request total KV-cache read. The peak decode throughput in Figure 4 is thus largely determined by the per-decode-token memory traffic in Tables 7 and 8. Models with smaller per-request KV-cache footprints can sustain larger batches, more effectively amortizing model-weight traffic in the memory-bound regime. Smaller per-request KV-cache traffic further reduces M_{decode} and amplifies this throughput advantage.

Interconnect sensitivity. Larger scale-up networking domains enable faster MoE token routing and are more beneficial when MoE routing traffic dominates. This aligns with the expected bottleneck shifting between attention and MoE. For prefill, computation dominates, so MoE routing traffic can be readily hidden at reasonably large batch sizes without substantially affecting interactivity. For decode, KV-cache traffic under CP is carried by aggregation collectives and runs at

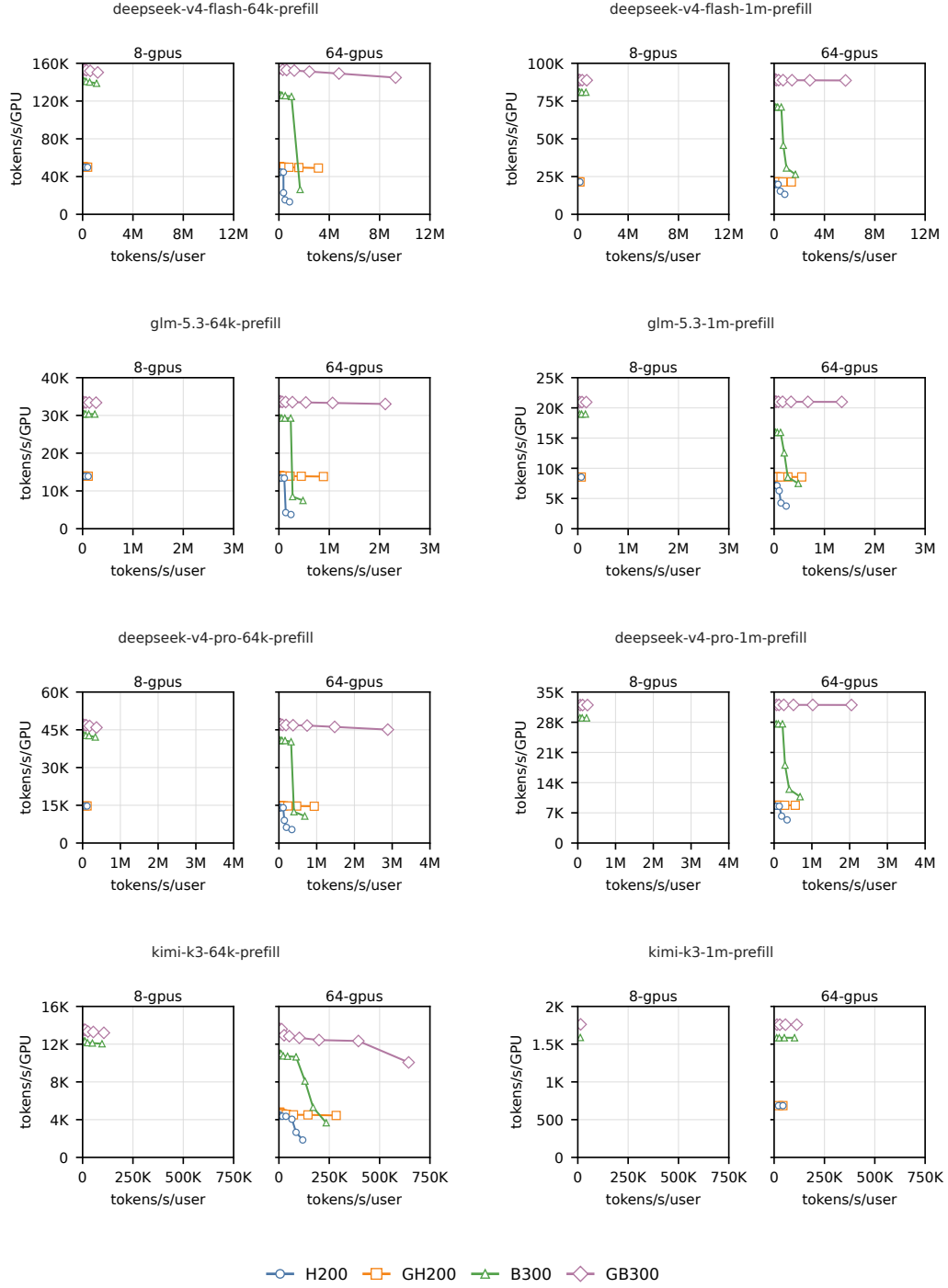


Figure 3: Throughput (token/s/GPU) - interactivity (token/s/user) pareto frontier for prefill.

similar bandwidth over scale-up and scale-out networks. As the KV cache grows, MoE token routing accounts for a smaller fraction of the total overhead because attention time increases and the routing traffic itself decreases with the lower sustainable batch size, leaving less end-to-end room for a larger scale-up networking domain to improve performance. The decode curves in Figure 4 follow the KV-cache traffic and batch sizes at peak throughput reported in Tables 7 and 8.

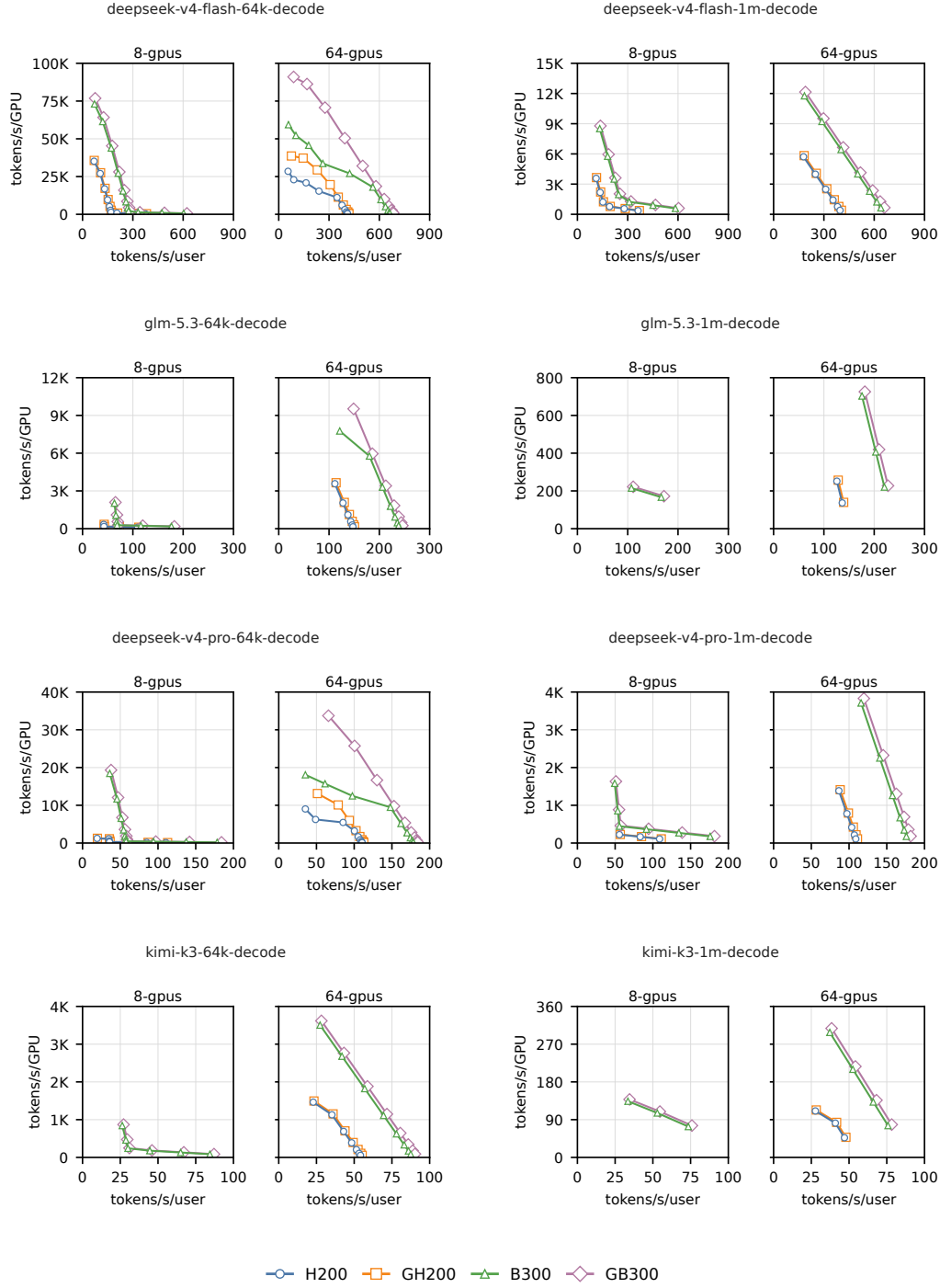


Figure 4: Throughput (token/s/GPU) - interactivity (token/s/user) pareto frontier for decode.

3.2 AI-DRIVEN AUTONOMOUS ARCHITECTING

3.2.1 SETUP

We implement a simple optimizer agent by running Claude Code persistently in non-interactive mode, without a preset turn limit (Anthropic, n.d.), thereby realizing the autonomous loop in Figure 1. Following DeepSeek’s official Claude Code integration guide, we configure DeepSeek V4 Pro

Table 7: Per-decode-token memory overhead at 64K context (64 NVIDIA GB300 GPUs).

Model	Per-request parameters traffic	Per-request KV-cache footprint	Per-request KV-cache traffic	Peak batch size	Per-decode-token memory traffic
DeepSeek V4 Flash	2.277 MiB	0.192 GiB	0.033 GiB	65,536	0.035 GiB
GLM-5.3	173.618 MiB	2.906 GiB	0.250 GiB	4,096	0.419 GiB
DeepSeek V4 Pro	24.811 MiB	0.275 GiB	0.055 GiB	32,768	0.079 GiB
Kimi K3	175.897 MiB	1.065 GiB	1.065 GiB	8,192	1.237 GiB

Table 8: Per-decode-token memory overhead at 1M context (64 NVIDIA GB300 GPUs).

Model	Per-request parameters traffic	Per-request KV-cache footprint	Per-request KV-cache traffic	Peak batch size	Per-decode-token memory traffic
DeepSeek V4 Flash	36.439 MiB	3.034 GiB	0.414 GiB	4,096	0.450 GiB
GLM-5.3	2,777.892 MiB	46.500 GiB	2.711 GiB	256	5.423 GiB
DeepSeek V4 Pro	396.984 MiB	4.344 GiB	0.608 GiB	2,048	0.996 GiB
Kimi K3	2,814.346 MiB	13.721 GiB	13.721 GiB	512	16.469 GiB

as the primary model and DeepSeek V4 Flash as the lightweight and subagent model ([DeepSeek-AI, n.d.](#)).

We let the optimizer agent improve the DeepSeek V4 Pro implementation in RoofLang. In each optimization session, we provide the agent with the RoofLang source repository, including the DeepSeek V4 Pro implementation, together with the baseline results from Section 3.1. The agent is instructed to keep seeking new device placements and semantics-preserving graph transformations that improve the throughput–interactivity Pareto frontier according to simulation feedback. It continues this loop until it decides to stop.

3.2.2 RESULTS

We discuss three representative cases below.

Cost- and head-aware contiguous PP partition. Balancing PP stages solely by layer count can leave the embedding-bearing first stage or output-head-bearing last stage on the critical path. The agent therefore calibrated stage costs from traces and enumerated legal contiguous partitions that reduced either or both boundary-stage loads without changing layer order or PP degree. For DeepSeek V4 Pro decode-64K on 256 B300 GPUs with batch 4,096, EP=16, and PP=16, it replaced the balanced partition ($4 \times 13, 3 \times 3$) with $(3, 4 \times 14, 2)$. Accounting for the extra embedding and output-head work reduced the maximum stage span and improved both throughput and interactivity by 20.3%.

DP–CP routed-expert splitting. The agent split each routed expert along the existing DP-major, CP-minor token shards and applied `graph.split_kernel` to the expert up and down projections, creating one copy per DP×CP shard. This allowed same-dimension Gather–Scatter pairs to be replaced by rank-local data edges, removing redundant local token materialization while preserving routing and expert ownership. Because the split incurs additional expert-weight reads, it was enabled only when each copy had at least 1,024 routed tokens and EP was at most eight. For DeepSeek V4 Pro prefill-8K on eight B300 GPUs with batch 512, the large prefill token volume amortized the extra weight traffic, and removing the local Gather–Scatter traffic improved both throughput and interactivity by 6.23%.

Node-local expert replication. The agent replicated selected experts across the two nodes of an EP=16 stage, so routed-token dispatch and combine paths used node-local NVLink rather than cross-node InfiniBand. The change preserves expert computation but trades additional expert-weight reads and HBM capacity for lower exposed routing communication, and was applied only to communication-bound cases. For DeepSeek V4 Pro prefill-1M on 16 B300 GPUs with batch 1 and CP=EP=16, the large routed-token shards made inter-node communication dominant; node-local routing reduced that bottleneck and improved both throughput and interactivity by 50.1%.

4 RELATED WORK

4.1 INFERENCE PERFORMANCE MODELING

Inference performance models span analytical abstractions and high-fidelity simulators. The Roofline model bounds performance using arithmetic intensity and hardware compute and bandwidth ceilings (Williams et al., 2009), while LLM-Viewer and Calculon apply analytical modeling to LLM inference (Yuan et al., 2024; Isaev et al., 2023). Simulation systems incorporate more execution detail: ASTRA-sim models distributed computation and communication, Vidur simulates LLM serving workloads, Frontier uses discrete-event simulation with timing and memory models calibrated against concrete serving stacks, and KernelSight-LM augments a kernel-level roofline model with learned efficiency factors and embeds it in a discrete-event serving simulator (Won et al., 2023; Agrawal et al., 2024; Feng et al., 2026; Yao et al., 2026). These methods trade evaluation cost and generality for different levels of fidelity, but performance estimation alone does not establish semantic equivalence between candidate architectures.

Verified graph optimization addresses this complementary requirement. Unity jointly optimizes algebraic transformations and parallelization for distributed DNN training using a unified parallel computation graph (Unger et al., 2022).

ROOFLANG combines a constrained mutation space with a roofline-based evaluator derived from analytical workload attributes and hardware specifications, without requiring profiled operator implementations. Its focus is this combination for AI-driven LLM inference system architecting, rather than transformation verification alone. Human verification remains necessary for the initial input graphs and the transformation contracts; subsequent exploration is restricted to placement operations and semantics-preserving transformations under their stated preconditions.

4.2 KERNEL AGENTS AND OTHER EFFORTS FOR RSI

Kernel agents and related efforts toward recursive self-improvement (RSI) close optimization loops around executable implementations (Schmidhuber, 2007; Zhang et al., 2026). SGLang exposes profiling tools and agent-facing developer skills (SGLang, n.d.; 2026), while Kernel Design Agents and KDA-Pilot organize repository-scale kernel optimization around task contracts, correctness gates, profiling evidence, and repeated benchmarking (MIT HAN Lab, 2026; BBuf, 2026). KernelBench and FlashInfer-Bench evaluate generated kernels through execution and performance tests (Ouyang et al., 2025; Xing et al., 2026), while GPU Kernel Scientist and KernelPro iteratively optimize kernels using profiling feedback (Andrews & Witteveen, 2025; Gai et al., 2026). AMD GEAK provides a related industrial agentic workflow (An et al., 2026). InferenceBench extends this approach to open-ended inference-server optimization (Yeon et al., 2026). OptiFlow optimizes collective communication by combining LLM-generated data-movement programs in a DSL, deterministic scheduling, and iterative refinement using real-hardware feedback (Long et al., 2026). These systems either manipulate real code directly or derive feedback from performance samples collected on real hardware, rather than using an implementation-independent architectural oracle.

This implementation-grounded feedback is concrete, but it couples the search space to the available kernels, compiler, framework, and hardware. A design may remain unreachable or appear inferior when its required features are absent or poorly optimized in the current stack. ROOFLANG decouples architectural design from concrete implementation details by evaluating graph organizations with hardware specifications and roofline relations. It can therefore search for designs that are intrinsically more efficient under theoretical resource limits before implementation quality biases the comparison. After an architecture is selected, kernel agents and framework-level tools can optimize its concrete implementation.

5 CONCLUSION, LIMITATIONS, AND FUTURE DIRECTIONS

ROOFLANG is an initial step toward enabling AI-driven architecting of LLM inference systems. The DSL combines expert-authored compute and hardware graphs, explicit placement and communication, semantics-preserving transformations, and a roofline-based discrete-event simulator. Its evaluation yields two central insights. First, DeepSeek V4-series models could achieve $3.5\text{--}39.5\times$

higher peak decode throughput than other representative models, despite no commensurate difference in total parameter counts. This advantage arises largely from compact KV-cache designs that support larger batches and reduce memory traffic. Second, a persistent optimizer agent discovered several new architectures that improved both throughput and interactivity of DeepSeek V4 Pro on NVIDIA B300 by 6.23–50.1%. Together, these findings indicate that a verification-constrained analytical model can expose useful model-level insights and guide architectural exploration before implementation-specific optimization.

Despite this progress, three limitations currently bound the autonomy, modeling fidelity, and scalability of ROOFLANG.

Initial-input verification. ROOFLANG currently assumes that the initial compute and hardware graphs are correct. This concentrates but does not eliminate the need for human verification. Future work should automate initial-input construction and validation, for example by deriving compute graphs from official baseline implementations and checking them against execution traces.

Modeling fidelity. The current models simplify behaviors that can materially affect performance. They assume ideal MoE load balance and omit KV-cache appending, speculative-decoding structures, and Kimi K3’s multimodal components. Future extensions should represent these behaviors explicitly and calibrate them against measured implementations while preserving the existing mutation and equivalence contracts.

Simulation scalability. The current discrete-event simulator does not scale efficiently to the largest configurations. Simulating Kimi K3 on 128 GPUs can require hours and hundreds of gigabytes of memory, which makes repeated agentic optimization impractical. Future work should reduce event and state materialization, reuse unchanged subgraphs across candidates, and introduce scalable parallel or approximate evaluation without weakening correctness checks.

REFERENCES

- Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation*, pp. 265–283, 2016. URL <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>.
- Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gula-vani, Ramachandran Ramjee, and Alexey Tumanov. Vidur: A large-scale simulation framework for LLM inference. In *Proceedings of Machine Learning and Systems*, volume 6, pp. 351–366, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/b74a8de47d2b3c928360e0a011f48351-Abstract-Conference.html.
- Zihao An, Yue Liu, Chao Xu, Ethan Yang, Hongtao Meng, Chushi Chen, Puyuan Yang, Subrahmanya Pavankumar Dubagunta, Saptarshi Majumder, Umang Pandey, Aditya Kumar Singh, Fan Wang, Zheng Yao, Qiong Zhu, Yixiong Huo, Dan Li, Ji Liu, Zeping Li, Ziqiong Liu, Arthur Huang, Fuwei Yang, Pratik Prabhanjan Brahma, Dong Li, Zicheng Liu, and Emad Barsoum. GEAK v4: From faster kernels to faster workloads. AMD Developer Central technical article, July 2026. URL <https://www.amd.com/en/developer/resources/technical-articles/2026/geak-v4.html>.
- Martin Andrews and Sam Witteveen. GPU Kernel Scientist: An LLM-driven framework for iterative kernel optimization. *arXiv preprint arXiv:2506.20807*, 2025. URL <https://arxiv.org/abs/2506.20807>.
- Anthropic. CLI reference. Claude Code documentation, n.d. URL <https://code.claude.com/docs/en/cli-usage>. Accessed September 3, 2026.

- BBuf. KDA-Pilot: Evidence-first autonomous GPU-kernel optimization campaigns for SGLang. GitHub repository, 2026. URL <https://github.com/BBuf/KDA-Pilot>. Accessed September 3, 2026.
- DeepSeek-AI. Integrate with Claude Code. DeepSeek API documentation, n.d. URL https://api-docs.deepseek.com/quick_start/agent_integrations/claude_code/. Accessed September 3, 2026.
- DeepSeek-AI et al. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024. URL <https://arxiv.org/abs/2412.19437>.
- DeepSeek-AI et al. DeepSeek-V4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026. URL <https://arxiv.org/abs/2606.19348>.
- Yicheng Feng, Xin Tan, Yangtao Deng, Yimin Jiang, Yibo Zhu, and Hong Xu. Frontier: Towards comprehensive and accurate LLM inference simulation. *arXiv preprint arXiv:2605.21312*, 2026. URL <https://arxiv.org/abs/2605.21312>.
- Jiading Gai, Shuai Zhang, Kaj Bostrom, Jin Huang, Vihang Patil, Haoyang Fang, Bernie Wang, Huzefa Rangwala, and George Karypis. Optimizing CUDA like a human: Micro-profiling tools as expert surrogates for LLM-based GPU kernel optimization. *arXiv preprint arXiv:2606.26453*, 2026. URL <https://arxiv.org/abs/2606.26453>.
- GLM-5-Team et al. GLM-5: From vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026. URL <https://arxiv.org/abs/2602.15763>.
- Roger W. Hockney. The communication challenge for MPP: Intel Paragon and Meiko CS-2. *Parallel Computing*, 20(3):389–398, 1994. doi: 10.1016/S0167-8191(06)80021-9.
- Mikhail Isaev, Nic McDonald, Larry Dennison, and Richard W. Vuduc. Calculon: A methodology and tool for high-level co-design of systems and large language models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–14, 2023. doi: 10.1145/3581784.3607102.
- Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. DeepSpeed Ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509*, 2023. URL <https://arxiv.org/abs/2309.14509>.
- Kimi Team et al. Kimi K3: Open frontier intelligence. *arXiv preprint arXiv:2607.24653*, 2026. URL <https://arxiv.org/abs/2607.24653>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023. doi: 10.1145/3600006.3613165.
- Seonghee Lee, Sungsoo Ha, Omri Almog, and Lucas Wilkinson. Efficient decode context parallelism with vllm for long context workloads. vLLM Blog, August 2026. URL <https://vllm.ai/blog/2026-08-07-decode-context-parallelism>.
- Dmitry Lepikhin, HyounJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020. URL <https://arxiv.org/abs/2006.16668>.
- Fei Long, Ziyue Yang, Kaihui Gao, Shuai Wang, Li Chen, Zhixiong Niu, Ran Shu, Wenxue Cheng, Peng Cheng, Yongqiang Xiong, and Dan Li. OptiFlow: Towards LLM-driven optimization of collective communication algorithms. In *Proceedings of the 10th Asia-Pacific Workshop on Networking*, pp. 73–81. ACM, 2026. doi: 10.1145/3820441.3820452.
- MIT HAN Lab. Kernel design agents. GitHub repository, 2026. URL <https://github.com/mit-han-lab/kernel-design-agents>. Accessed September 3, 2026.

- Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–15, 2021. doi: 10.1145/3458817.3476209.
- NVIDIA. NVIDIA GB300 NVL72. Product specifications, 2025. URL <https://www.nvidia.com/en-us/data-center/gb300-nvl72/>. Accessed September 3, 2026.
- NVIDIA. Grace performance tuning guide. Technical guide, 2026a. URL <https://docs.nvidia.com/dccpu/grace-perf-tuning-guide/index.html>. Accessed September 3, 2026.
- NVIDIA. NVIDIA HGX AI Factory: NVIDIA Enterprise Reference Architecture. Reference architecture white paper, 2026b. URL <https://docs.nvidia.com/enterprise-reference-architectures/hgx-ai-factory/latest/index.html>. Accessed September 3, 2026.
- NVIDIA. NVIDIA H200 GPU. Product specifications, n.d. URL <https://www.nvidia.com/en-us/data-center/h200/>. Accessed September 3, 2026.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 47356–47415, 2025. URL <https://proceedings.mlr.press/v267/ouyang25a.html>.
- Jürgen Schmidhuber. Gödel machines: Fully self-referential optimal universal self-improvers. In Ben Goertzel and Cassio Pennachin (eds.), *Artificial General Intelligence*, pp. 199–226. Springer, 2007. doi: 10.1007/978-3-540-68677-4_7.
- SGLang. SGLang .claude/skills. SGLang GitHub repository, commit d3bae71, 2026. URL <https://github.com/sgl-project/sglang/tree/d3bae71e3f1fd983853cae4f95591d4a8545325f/.claude/skills>. Accessed September 3, 2026.
- SGLang. Benchmark and profiling. SGLang documentation, n.d. URL https://docs.sglang.io/docs/developer_guide/benchmark_and_profiling. Accessed September 3, 2026.
- Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 31094–31116, 2023. URL <https://proceedings.mlr.press/v202/sheng23a.html>.
- Colin Unger, Zhihao Jia, Wei Wu, Sina Lin, Mandeep Baines, Carlos Efrain Quintero Narvaez, Vinay Ramakrishnaiah, Nirmal Prajapati, Pat McCormick, Jamaludin Mohd-Yusof, Xi Luo, Dheevatsa Mudigere, Jongsoo Park, Misha Smelyanskiy, and Alex Aiken. Unity: Accelerating DNN training through joint optimization of algebraic transformations and parallelization. In *16th USENIX Symposium on Operating Systems Design and Implementation*, pp. 267–284. USENIX Association, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/unger>.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.
- William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. ASTRA-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software*, pp. 283–294, 2023. doi: 10.1109/ISPASS57527.2023.00035.

- Shanli Xing, Yiyang Zhai, Alexander Jiang, Yixin Dong, Yong Wu, Zihao Ye, Charlie Ruan, Yingyi Huang, Yineng Zhang, Liangsheng Yin, Aksara Bayyapu, Luis Ceze, and Tianqi Chen. FlashInfer-Bench: Building the virtuous cycle for AI-driven LLM systems. *arXiv preprint arXiv:2601.00227*, 2026. URL <https://arxiv.org/abs/2601.00227>.
- Xiteng Yao, Taeho Kim, Hengzhi Pei, Xinle Liu, Kyle Ulrich, Leonard Lausen, Ashish Khetan, Xiang Song, George Karypis, and Martin Herboldt. KernelSight-LM: A kernel-level LLM inference simulator. *arXiv preprint arXiv:2606.28565*, 2026. doi: 10.48550/arXiv.2606.28565. URL <https://arxiv.org/abs/2606.28565>.
- Jehyeok Yeon, Ben Rank, and Maksym Andriushchenko. InferenceBench: A benchmark for open-ended LLM inference optimization by AI agents. *arXiv preprint arXiv:2607.20468*, 2026. URL <https://arxiv.org/abs/2607.20468>.
- Zhihang Yuan, Yuzhang Shang, Yang Zhou, Zhen Dong, Zhe Zhou, Chenhao Xue, Bingzhe Wu, Zhikai Li, Qingyi Gu, Yong Jae Lee, Yan Yan, Beidi Chen, Guangyu Sun, and Kurt Keutzer. LLM inference unveiled: Survey and roofline model insights. *arXiv preprint arXiv:2402.16363*, 2024. URL <https://arxiv.org/abs/2402.16363>.
- Z.ai. GLM-5.3. Hugging Face model card, 2026. URL <https://huggingface.co/zai-org/GLM-5.3>. Accessed September 3, 2026.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://iclr.cc/virtual/2026/poster/10007327>.
- Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pp. 193–210, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.